

Assignment 3 Report

Evan Ellis, Ayaan Haque, Erfan Jahanparast

QA Dataset Creation

To create the QA test dataset, we browsed the EECS website and, whenever we found a page that had factual information, wrote down a number of questions from that page. For every 20 questions we included one Yes/No question and one open-ended question. For the first half of the questions, we focused on getting questions from the narrative about pages. For the second half, we focused on questions from list-style pages, like the list of professors or tech reports. We generated 100 questions. We had two group members label the same set of 30 questions and got an average token level F1 of 0.8123, a kappa of 0.6463, and an alpha of 0.6503. We mostly agreed on the facts in the labels, but sometimes differed on the wording of the response. Here are a few sample questions:

Question	Answer	URL
What CS program aims to support students from diverse backgrounds?	CS Scholars	https://eecs.berkeley.edu/cs-scholars/
Who is the faculty advisor of CS Scholars?	Armando Fox	https://eecs.berkeley.edu/cs-scholars/
Who did Cade Gordon submit his technical report with?	Amy Lu and Pieter Abbeel	https://www2.eecs.berkeley.edu/Pubs/TechRpts/2025/

Retrieval Corpus

To construct the retrieval corpus, our `build_corpus.py` script first runs BFS on the EECS website to construct a list of URLs. This BFS runs to a maximum depth of 3 and a maximum URL count of 6000. We then fetch the pages in parallel. We had some difficulty with fetching pages from the `ww2` urls because they were more likely to timeout, so we added an exponential backoff on the fetch. Even still, some URLs timed out while fetching. We store the corpus as a jsonl with one entry per page. Initially, we had tried creating chunks while building the corpus, but this was problematic because to iterate on the chunking we had to re-build the corpus from scratch. We do, however, linearize the page at this step. Our linearization first looks for one of the common CSS tags we found that demarcated the start of the main page content. It then removes noise tags like `<script>`, `<style>`, etc. We then extract tables, and then convert any heading tags to `##` in markdown. Any non-heading tag has its contents appended as-is. Finally, pages with fewer than 60 characters are discarded.

We iterated on this corpus by manually inspecting the returned pages and ensuring that they were linearized correctly. We specifically looked for broken text or leftover HTML content that would make retrieval more difficult. This led to a total corpus size of 1,257 pages. On the dev set using our corpus, our RAG system had 41% exact match, and a token F1 of .4431. On the dev set using the official corpus, our RAG system had a 45% exact match, and a token F1 of 0.5243. The official corpus had far more pages than ours at 4,753. A lot of these seem to come from numbered pages, like the EECS news and tech reports. Our corpus didn't capture all of these pages, despite our attempt to.

Rag System

Our RAG system combines two techniques, bm25 ranking and embedding ranking. It also uses a two stage retrieval method: first, we find the most relevant documents, then we find the most relevant sections from those documents. For each question, we get the sparse bm25 scores and compute the similarity between the embedded query and the page embeddings. We then use reciprocal rank fusion which sums the smoothed reciprocals of the ranks from bm25 and the embedding to get a single score. We use a smoothing of $1/(\text{rank} + 60)$ so that small changes in rank don't dominate the final score. We then get the top 10 pages. If the top 10 pages have extremely low similarity, we return [] so that the result will be UNKNOWN—we don't want to give an answer when there are no relevant documents. To construct the final context, we split the pages into sections delineated by their markdown headers, and rank the sections by the number of query words that appear in them. We then pack these sections up to 12,000 characters into the context of the LLM. Finally, we query Llama 3.1 8B Instruct with the question and the retrieved context.

Ablations

When designing this system, we iterated on several failure modes we identified in our first attempts. We tried just using a sparse rankings with no embedding model. On the official corpus and dev set, this gave us a 44% exact match and a 0.5064 Token F1, so slightly lower. Just using the embedding scores was even worse, with a 38% exact match and a 0.4484 Token F1. This is somewhat unexpected, because I would expect a pure embedding ranking to perform better than bm25. Our sentence encoder, sentence-transformers/all-MiniLM-L6-v2, could be too weak for the task. We also initially tried feeding each top-ranked page sequentially, rather than splitting up by sub-heading. However, many pages are very long, and this led to degraded performance because irrelevant context was being included and crowding out the most useful information. This had a 40% exact match and a Token F1 of 0.4617.

Error Analysis

To perform error analysis, we added several functions that checked whether we a) retrieved the gold URL, b) the gold URL is even in the corpus, and c) the gold answer was contained in the text. A very common type of error is slightly different formatting by the LLM than by the gold answer. For example, Sloan Research Fellowship versus Sloan Research Fellow or “11:00am–2:00pm, Monday–Friday” versus “11:00 am – 2:00 pm.” The latter has an F1 score of 0, so the metric is failing. A better metric might be to ask an LLM if the answers are equivalent, or to normalize the outputs before computing the metric. Another type of error is the LLM poorly reading its context, for example our model said the 5th year master's was 2 years, even though the correct answer was in its context. Finally, the most common type of error is that the URL is not retrieved by our ranking system.

Takeaways and Future Ideas

We benefited a lot from finding the right metrics and building a good amount of observability into our pipeline. This takeaway generalizes to lots of ML work, where aggregate numbers are not sufficient to debug a pipeline. In the future, we would like to pre-process the text with an LLM to create a bank of facts, rather than walls of text that need to be retrieved. This would greatly improve our recall of the correct URL, and make the context easier for the LLM to parse.

Contribution Statement

Ayaan built the initial RAG system. Evan worked on the scrapper, the RAG system, and the report. Erfan built our test set and iterated on the RAG system.

GenAI Statement

We acknowledge the use of Cursor for limited tab completion. We used the outputs to explain rank_bm25 and some BeautifulSoup functions. We have the ability to explain and even independently replicate the work done in this document if asked by an instructor